Regularisation and the bias–variance trade-off

Why more capacity stops helping, and what to do about it

LECTURE 5 | GÉRON CH. 4

One identity, three terms, and the one line that repairs an ill-posed fit

---

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Worked example: the Titanic passenger manifest

Where we are

The previous lecture fitted a logistic regression to 712 Titanic passengers and then added polynomial capacity, recording **both** curves at every degree:

Degree	Columns	Training log loss	Held-out log loss
1	22	0.395	0.468
2	32	0.381	**0.467 ✓**
3	52	0.355	0.538
5	143	0.286	**✗ 1.922**

One curve falls forever. The other turns and climbs.

Two questions about that table

1. The held-out error is 0.468 and the training error is 0.395. **Where does the difference come from?**
2. 0.468 against an anchor of 0.666. **How far down could it ever go?**

> **WHY BOTH ANSWERS MATTER**
>
> The two questions have different repairs, and the repairs point in opposite directions. Attack the wrong one and you spend an afternoon making the model worse.

Today

min	What
0–10	where we are, and the two questions the sweep cannot answer
10–30	**the mathematics** — the bias–variance decomposition, derived
30–70	**the method** — measuring the three terms, reading learning and validation curves, ridge, lasso, elastic net, early stopping
70–85	choosing a penalty honestly, and the test set once
85–90	the notebook

The decomposition and what each expectation is over; reading a diagnosis off a pair of curves; the three penalties and what each does to the coefficients. Solver names, wall-clock timings, and the Titanic feature engineering.

EXAMINABLE

NOT EXAMINABLE

THE MATHEMATICS

The bias–variance decomposition

Géron §4.4 — the identity that says what a generalisation error is made of

20 minutes · examinable

First: what is random here?

Not the passenger. Fix one passenger x and ask about them repeatedly.

- The **training set** D is random — you happened to get these 712 rows out of the 891
- The **label** y of the passenger is random — two people with identical recorded values did not both survive

Every expectation in this derivation is over those two sources and nothing else. The model is deterministic given D.

Three numbers for one passenger

Symbol	Meaning	Observed?
$\hat{p}_D(x)$	what *your* model predicts, having seen D	**yes ✓**
$\bar{p}(x) = \mathbb{E}_D[\hat{p}_D(x)]$	the average prediction over all training sets you could have drawn	**no — estimated ✗**
$p^*(x)$	the true probability that this passenger survives	**never ✗**

$y \in \{0, 1\}$ is what actually happened. It is never p^*, and that gap becomes the third term.

The quantity to decompose

$$\mathbb{E}_{D,\,y}\left[\left(y - \hat{p}_D(x)\right)^2\right]$$

EXPECTED SQUARED ERROR AT ONE PASSENGER

Averaged over passengers this is the **Brier score** the previous lecture already reported. We decompose it at a single x and then average — the decomposition is linear in x, so nothing is lost by doing it one passenger at a time.

Step 1 · Add and subtract the average prediction

Hold y fixed and average over D only:

$$(y - \hat{p}_D)^2 = \Big(\underbrace{y - \bar{p}}_{\text{does not depend on } D} + \underbrace{\bar{p} - \hat{p}_D}_{\text{mean } 0} \Big)^2$$

Expand the square. The cross term is $2\,(y - \bar{p})\,\mathbb{E}_D[\bar{p} - \hat{p}_D] = 0$, because $\bar{p}$ is *defined* as $\mathbb{E}_D[\hat{p}_D]$.

Step 2 · Two terms, exactly

$$\mathbb{E}_D\left[(y - \hat{p}_D)^2\right] = (y - \bar{p})^2 + \underbrace{\mathbb{E}_D\left[(\hat{p}_D - \bar{p})^2\right]}_{\textbf{variance}}$$

The second term is the variance of your predictor. It says how much the answer moves when the 712 rows move — and it does not mention the truth at all.

✓ An identity, not an approximation. We check it numerically later in this lecture, and the residual is 2.8×10^{-17}.

Step 3 · Now average over the label as well

Repeat the trick on $(y - \bar{p})^2$, this time inserting p^*:

$$\mathbb{E}_y\left[(y - \bar{p})^2\right] = \underbrace{(p^* - \bar{p})^2}_{\textbf{squared bias}} + \underbrace{p^*(1 - p^*)}_{\textbf{noise}}$$

Same cancellation: $\mathbb{E}_y[y] = p^*$, so the cross term vanishes again. And $\mathrm{Var}(y) = p^*(1 - p^*)$ because y is Bernoulli.

The decomposition

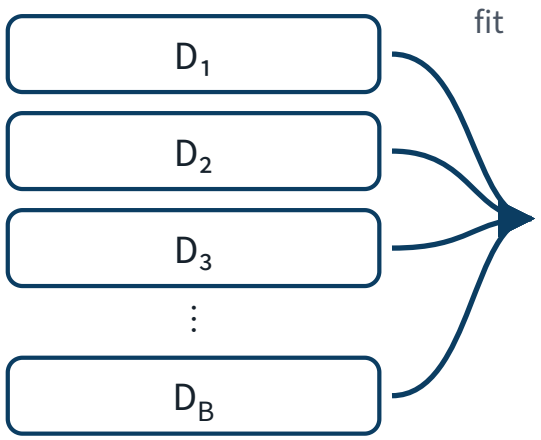
$$\mathbb{E}\left[(y - \hat{p}_D)^2\right] = (p^* - \bar{p})^2 + \mathbb{E}_D\left[(\hat{p}_D - \bar{p})^2\right] + p^*(1 - p^*$$

Term	Means	Cured by
squared bias	wrong *on average* — the model family cannot reach the truth	more capacity
variance	unstable — a different sample gives a different answer	less capacity, or more data
noise	the label is not a function of the recorded columns	✗ **nothing**

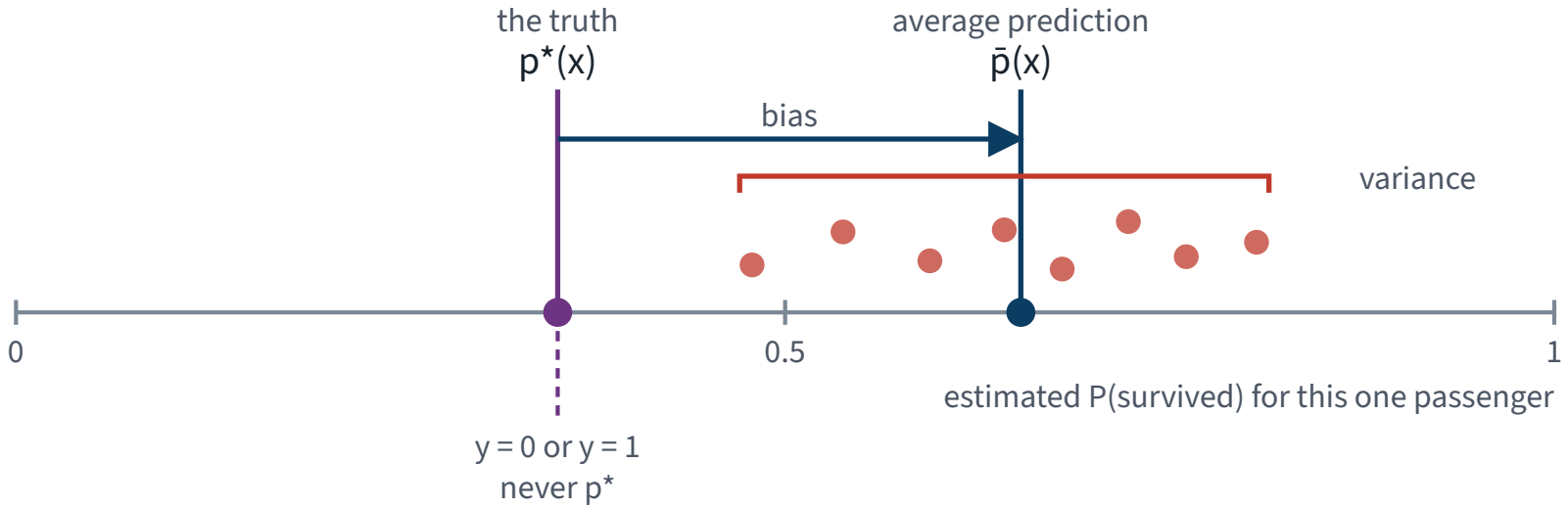
✗ The first two repairs point in opposite directions. That is the whole difficulty, and it is why it is called a trade-off.

The three terms, in one picture

One passenger, every training set you could have drawn



712 rows — but which 712?



$$E\,[(y - \hat{p})^2] = \mathbf{(p^* - \bar{p})^2} + \mathbf{E\,[(\hat{p} - \bar{p})^2]} + \mathbf{p^*(1 - p^*)}$$

total error — squared bias — wrong on average — variance — unstable — noise — nothing to be done

Bias is the distance from the centre of the spread to the truth. Variance is the width of the spread. Neither is visible from one training set.

The derivation, in one page

1. **1** $\mathbb{E}_{D,y}[(y - \hat{p}_D)^2]$ at one fixed x

   the quantity; the randomness is D and y, never the test point

2. **2** $\bar{p} := \mathbb{E}_D[\hat{p}_D]$

   the pivot: the average prediction over training sets

3. **3** $(y - \hat{p}_D)^2 = \big((y - \bar{p}) + (\bar{p} - \hat{p}_D)\big)^2$

   add and subtract $\bar{p}$; the cross term has mean 0 by the definition of $\bar{p}$

4. **4** $\mathbb{E}_D[\,\cdot\,] = (y - \bar{p})^2 + \mathbb{E}_D[(\hat{p}_D - \bar{p})^2]$

   the second term is the **variance**

5. **5** $\mathbb{E}_y[(y - \bar{p})^2] = (p^* - \bar{p})^2 + p^*(1 - p^*)$

   the same trick on p^*; $\mathrm{Var}(y) = p^*(1 - p^*)$ for a Bernoulli

6. **6** total $=$ bias^2 $+$ variance $+$ noise

   exact, for squared error, at every x — and linear, so it averages

An honest caveat before we measure anything

The identity is exact for **squared error**. Our headline metric is log loss, and log loss does *not* split into three additive terms like this.

> **WHAT WE DO ABOUT IT**
>
> The decomposition is measured on the **Brier score** — which the previous lecture already reported alongside log loss, and which is the squared error of a probability. The *shape* of the answer transfers; the numbers are Brier numbers and are labelled as such.

Saying “bias–variance” about a log loss curve is common and slightly wrong. Doing it knowingly, and saying which metric you decomposed, is the difference.

What the derivation buys you

The result	What it lets you do
Three additive terms	ask *which* one is large before choosing a repair
Variance does not mention p^*	estimate it without knowing the truth — refit and look at the spread
Noise depends only on x	measure a **floor** that no model of these columns can beat
✓ The gap between two curves	**✓ read a diagnosis off a plot in ten seconds**

✓ The rest of this lecture is that table, executed: measure all three terms on the pipeline of the previous lecture, then repair the one that turned out to be large.

THE METHOD

Measure all three, on your own pipeline

40 minutes · examinable

Start with the term you cannot fix

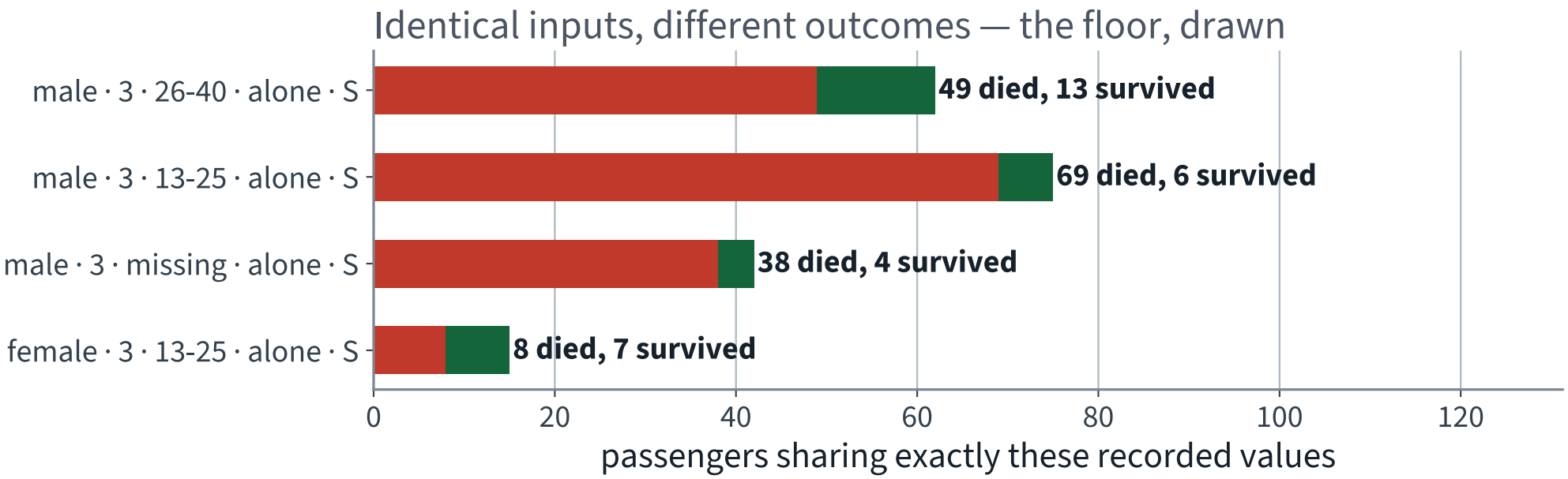
p^* is never observed — but where several passengers share *exactly* the same recorded values, every model of those columns must give them the same number. Whatever their outcomes do inside that cell is a floor.

```
keys  = ["Sex", "Pclass", "AgeBand", "FamBand", "Embarked"]
cells = full.groupby(keys, observed=True)["Survived"].agg(["sum", "count"])

# k(m-k)/(m(m-1)) is the unbiased estimator of p(1-p) from m Bernoulli draws
multi = cells[cells["count"] >= 2]
var   = multi["sum"] * (multi["count"] - multi["sum"]) / (
        multi["count"] * (multi["count"] - 1))
noise = (var * multi["count"]).sum() / multi["count"].sum()
```

133 distinct cells; 102 of them hold at least two passengers, covering 858 people.

Identical inputs, different outcomes



✗ 62 third-class men aged 26 to 40, travelling alone from Southampton, with every recorded value identical. 13 of them survived. No model of these columns can separate them.

The floor

0.121

Brier score. Measured, not assumed.

56 of the 102 shared cells are *mixed* — 657 passengers sit in a cell where the outcome went both ways. That is 74% of everyone on board, living inside a cell whose answer cannot be certain.

It is an *under*-estimate of the true noise: two passengers in the same cell may still differ in ways the manifest never recorded. The real floor is at least this high.

Now the other two terms — by resampling D

```
rng   = np.random.default_rng(42)
draws = [rng.choice(len(X_train), size=400, replace=False)
         for _ in range(200)]

P = np.array([pipeline(degree=d).fit(X_train.iloc[s], y_train.iloc[s])
                                .predict_proba(X_test)[:, 1]
              for s in draws])                          # 200 x 179

pbar     = P.mean(axis=0)
variance = ((P - pbar) ** 2).mean(axis=0)
bias2_pn = (y_test.values - pbar) ** 2                  # bias^2 + noise, together
total    = ((y_test.values - P) ** 2).mean(axis=0)
```

200 training sets of 400 rows each, drawn from the 712 we hold. This is the expectation over D, done by brute force — which is the only way to see a quantity defined over training sets you did not draw.

Bias and noise cannot be separated without p^*, so they are reported as one column. The floor above is what splits them.

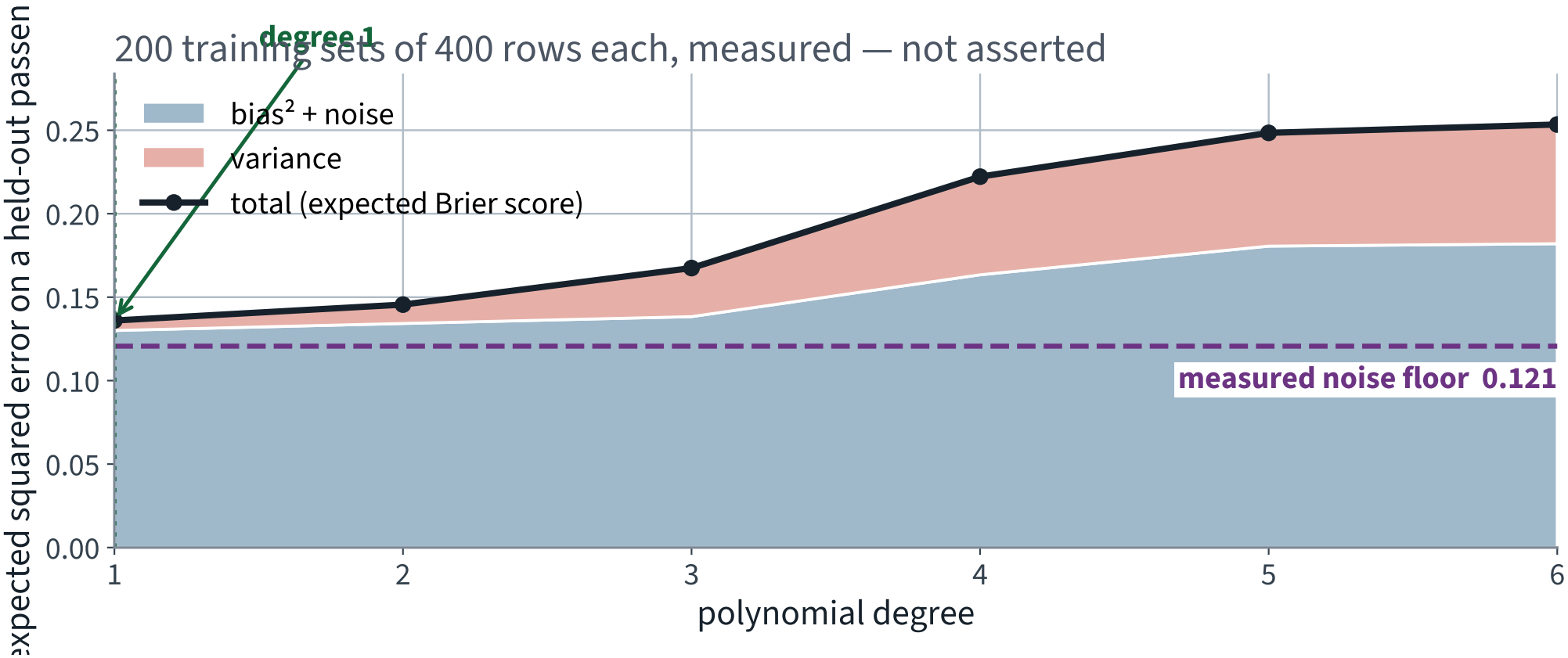
Check the identity before believing the plot

```
>>> abs(total.mean() - variance.mean() - bias2_pn.mean())
2.7755575615628914e-17
```

Machine epsilon on a double. The three columns you are about to see are not a *model* of the error — they *are* the error, rearranged.

✓ Do this every time you implement a decomposition. If the residual is not at floating-point noise you have a bug, and the picture will still look plausible.

The degree sweep, taken apart



Between degree 1 and degree 6 the variance term is multiplied by **twelve**. Everything else is nearly flat.

The same figure, as numbers

Degree	bias² + noise	variance	total
1	0.130	**0.006 ✓**	**0.136 ✓**
2	0.134	0.011	0.146
3	0.138	0.029	0.167
4	0.163	0.059	0.222
5	0.181	**✗ 0.068**	**✗ 0.248**
6	0.182	**✗ 0.071**	**✗ 0.253**

Read the first column: 0.130 at degree 1, against a measured noise floor of 0.121. **Almost all of it is noise.** The squared bias of a degree-1 logistic model on these features is under 0.01.

✗ So there was never much bias to buy back. Every extra degree bought variance and nothing else.

Now put it back on the two curves

The training curve

Falls forever. It is measured on the rows that were fitted, so it sees no variance term at all — the model was tuned to those exact rows.

The held-out curve

Turns at degree 2 and climbs. It is the sum of all three terms, and from degree 3 the variance term dominates.

THE IDENTIFICATION, STATED ONCE

The vertical gap between the two curves is the variance term. Degree 1: gap 0.073. Degree 5: gap 1.636. The gap grew by a factor of 22 while nothing about the data changed.

Two shapes, two diagnoses

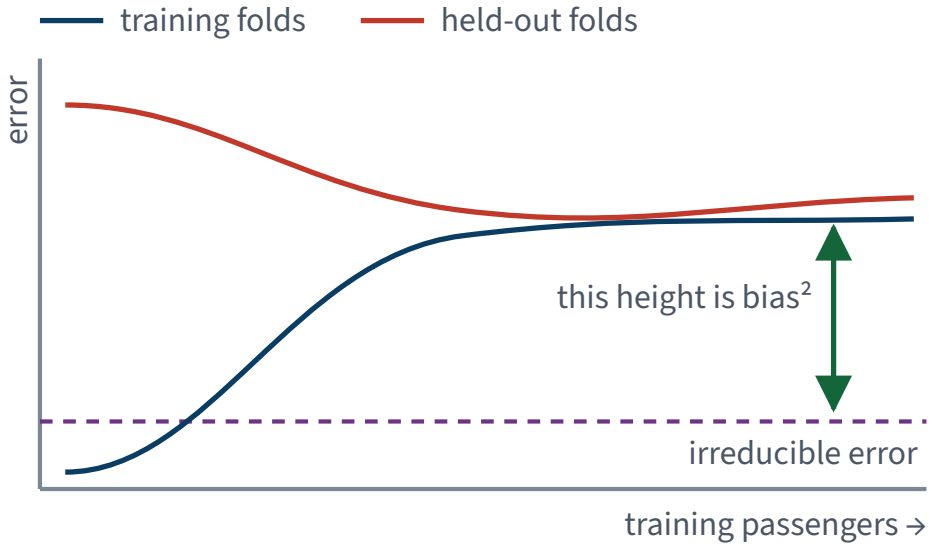
- Both curves plateau **high and together** → bias. The model is too rigid; more data changes nothing
- A gap that is **still open** at the last point → variance. More data would help, and you may not be able to get any

These are the only two shapes, and telling them apart takes ten seconds once you know what you are looking at.

The two shapes, drawn

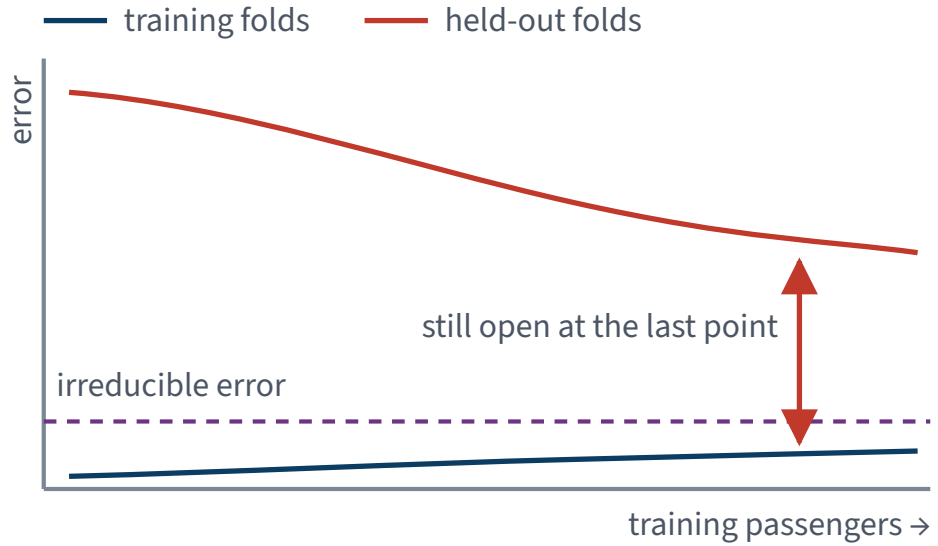
Two shapes, two diagnoses, two different repairs

Both curves plateau high, and together



Bias. More data will not move either curve.
Repair: more features, higher degree, weaker penalty.

A gap that does not close



Variance. More data would help, and is not on offer.
Repair: shrink the model — ridge, lasso, early stopping.

The repairs are on the diagram because they are opposites. Applying the left-hand repair to a right-hand curve is the most common wasted afternoon in applied machine learning.

A learning curve: the x-axis is *rows*

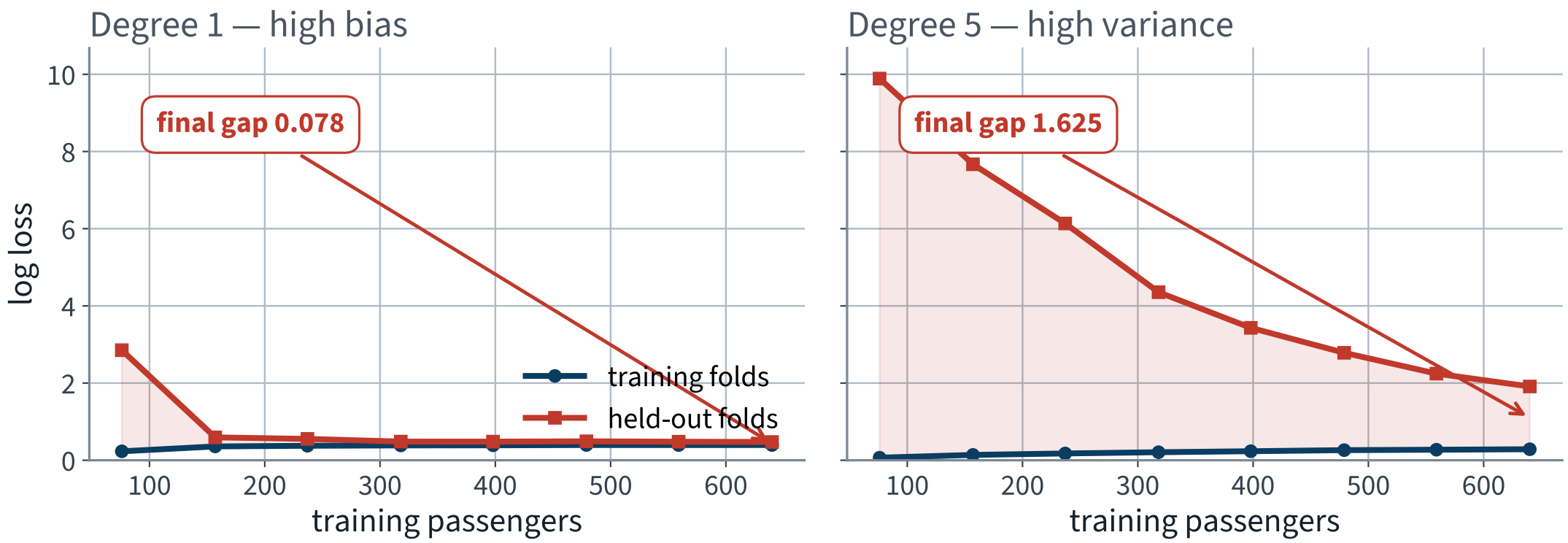
```
from sklearn.model_selection import learning_curve

n, train, valid = learning_curve(
    pipeline(degree=d), X_train, y_train, cv=cv,
    train_sizes=np.linspace(0.12, 1.0, 8),
    scoring="neg_log_loss", shuffle=True, random_state=42)
```

Same two curves as the degree sweep, a different knob — and the shape answers a question the degree sweep cannot: *would more data help?*

Note `shuffle=True`. Without it the sub-samples are the first n rows of the frame, in whatever order the frame happens to be in.

Degree 1 and degree 5, as the data grows



Left: the curves have met. Right: they have not, and the gap is still falling — the model is starved of rows, not of capacity.

Reading those two panels

	Degree 1	Degree 5
Held-out log loss at 76 rows	2.853	✗ 9.889
Training log loss at 640 rows	0.395	0.286
Held-out log loss at 640 rows	0.474	✗ 1.911
Gap at 640 rows	**0.078 ✓**	✗ 1.625

The degree-5 held-out curve falls from 9.9 to 1.9 as rows arrive, and is still falling steeply at 640. Extrapolating, this model would need *several times* our dataset before it became competitive.

✓ There are 891 passengers on the Titanic. There will never be more. Rows are not a knob we can turn, so the only repair left is to shrink the model.

The other curve: score against a *hyperparameter*

Curve	x-axis	Answers
Learning curve	training-set size	would more *rows* help?
Validation curve	one hyperparameter	which *setting* is right, and is the optimum interior?

```
from sklearn.model_selection import validation_curve

train, valid = validation_curve(
    pipeline(degree=5), X_train, y_train, cv=cv,
    param_name="clf__C", param_range=np.logspace(-4, 4, 17),
    scoring="neg_log_loss")
```

✗ Read the *ends* of a validation curve first. A minimum sitting on the edge of the range means the range was too narrow and the answer you read off it is an artefact of where you stopped looking.

Why it is called a trade-off

$$\text{total} = \underbrace{\text{bias}^2}_{\downarrow \text{ with capacity}} + \underbrace{\text{variance}}_{\uparrow \text{ with capacity}} + \underbrace{\text{noise}}_{\text{fixed}}$$

You are minimising a sum of two terms that move in opposite directions. The minimum is interior, and on this dataset it sits at degree 1 or 2 — exactly where the held-out curve turned.

The word “trade-off” is doing real work. There is no setting that is good at both.

DIAGNOSIS

What went wrong at degree 5

Three faults, and one term repairs all three

The suspect

1.922

Degree 5 is **almost three times worse than saying nothing at all**.

Not worse than the best model. Worse than the anchor of 0.666 — worse than a model that has learned only how many people died.

How is that even possible?

Held-out accuracy at degree 5 was 76.0% — poor, but not absurd. Log loss went to 1.922. The two metrics disagree because they measure different things.

$$-\log(0.01) \approx 4.6 \qquad \text{one confident mistake}$$

A single passenger given probability 0.01 who then survives contributes 4.6 to the average on its own. Accuracy records that passenger as one error and moves on.

✓ Log loss is unbounded above. That is not a defect — it is the metric saying that confident-and-wrong is a different failure from wrong.

Diagnosis 1 · Variance, straight off the decomposition

	Degree 1	Degree 5
Columns	22	✗ 143
Training rows	712	712
Rows per weight	32	✗ 5
Variance term, Brier	0.006	✗ 0.068

Five rows per weight. Move one passenger between folds and the fitted surface moves with them.

This is the diagnosis the derivation was built to deliver, and it is not yet the whole story. Two more faults are underneath it.

Diagnosis 2 · The minimiser does not exist

Suppose that in the expanded feature space some $\boldsymbol{\theta}$ separates the classes perfectly. Then

$$\sigma(c\,\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}) \;\longrightarrow\; \begin{cases} 1 & \text{if } \boldsymbol{\theta}^{\mathsf{T}}\mathbf{x} > 0 \\ 0 & \text{if } \boldsymbol{\theta}^{\mathsf{T}}\mathbf{x} < 0 \end{cases} \qquad \text{as } c \to \infty$$

so the training log loss falls monotonically towards zero as $\|\boldsymbol{\theta}\| \to \infty$. The infimum is never attained.

Degree	1	2	3	4	5	6
Converged	yes ✓	yes ✓	yes ✓	✗ no	✗ no	✗ no
Largest $\lvert\theta_j\rvert$	4.87	5.43	6.32	✗ 18.70	✗ 11.07	3.98

Diagnosis 3 · The minimiser is not unique

Recall from the previous lecture that `FamilySize = SibSp + Parch + 1` exactly, on every row. Standardise the five numeric columns, form $\mathbf{X}^\top\mathbf{X}$, and look at its spectrum:

```
>>> np.linalg.eigvalsh(Z.T @ Z)
array([1.81e-12, 4.29e+02, 5.43e+02, 7.90e+02, 1.80e+03])
```

The smallest eigenvalue is zero to the precision the arithmetic allows. Four columns of information in five columns of data.

✗ Condition number about 9×10^{14}. A double carries about 10^{16} of relative precision, so roughly two significant digits survive the solve.

What adding $\alpha\mathbf{I}$ does to that spectrum

If $\mathbf{X}^\intercal\mathbf{X}\,\mathbf{v} = \lambda\mathbf{v}$, then

$$(\mathbf{X}^\intercal\mathbf{X} + \alpha\mathbf{I})\,\mathbf{v} \;=\; (\lambda + \alpha)\,\mathbf{v}$$

- Same eigenvectors; every eigenvalue moved up by exactly α
- $\mathbf{X}^\intercal\mathbf{X}$ is positive semi-definite, so $\lambda \geq 0$, so $\lambda + \alpha \geq \alpha > 0$
- No eigenvalue is zero, so **the matrix is invertible for every $\alpha > 0$**

LECTURE 2, CLOSED

Lecture 2 derived $\hat{\boldsymbol{\theta}} = (\mathbf{X}^\intercal\mathbf{X})^{-1}\mathbf{X}^\intercal y$ and left open what to do when that inverse does not exist. **Adding $\alpha\mathbf{I}$ makes it exist, for every $\alpha > 0$, whatever the columns do.** Those three lines are the proof.

and it bounds the conditioning

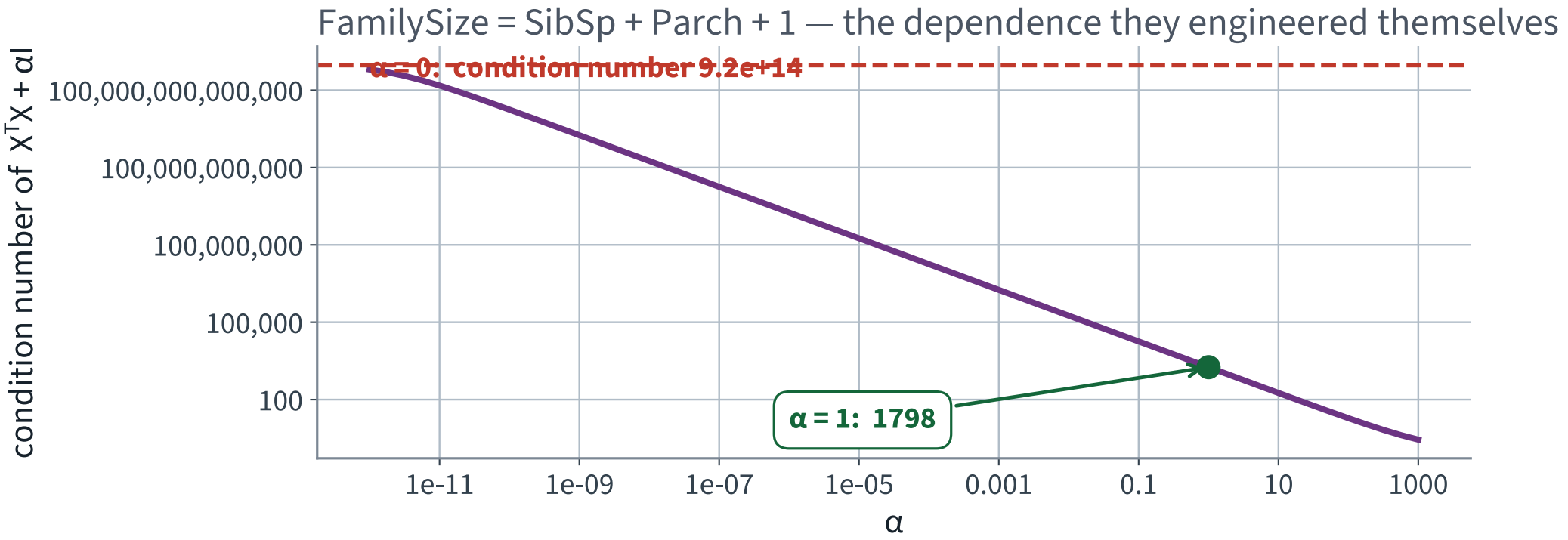
$$\kappa(\mathbf{X}^\top\mathbf{X} + \alpha\mathbf{I}) = \frac{\lambda_{\max} + \alpha}{\lambda_{\min} + \alpha} \leq \frac{\lambda_{\max} + \alpha}{\alpha}$$

The bound on the right does not mention $\lambda_{\min}$ at all. Whatever the data do — exact collinearity included — the conditioning is controlled by a number you choose.

✓ At $\alpha = 1$ the condition number of our singular matrix falls from 9×10^{14} to 1798.

And κ is what set the step count for gradient descent in the previous lecture. The penalty is not only a statistical device; it is what makes the optimisation tractable.

Condition number against α



A straight line of slope -1 over twelve decades: in this regime the condition number is simply $\lambda_{\max}/\alpha$. The dependence in the data is not repaired — it is *dominated*.

Three faults, one term

Fault	Evidence	A statement about
Variance	held-out minus training gap of 1.636 at degree 5	the estimator
No minimiser	lbfgs stops at the iteration cap from degree 4 up	the objective
No unique minimiser	rank 23 of 29; an eigenvalue at 10^{-12}	the design matrix

✓ One term repairs all three, and it is the same term in every case.

THE METHOD

Regularisation

Ridge, lasso, elastic net, and stopping early

The idea, in one sentence

Stop asking for the best fit. Ask for the best fit *among small models*.

$$J(\boldsymbol{\theta}) = \underbrace{L(\boldsymbol{\theta})}_{\text{fit the data}} + \underbrace{\alpha\,\Omega(\boldsymbol{\theta})}_{\text{stay small}}$$

Ω penalises the size of the weights; α says how much you care. Setting $\alpha = 0$ recovers exactly the previous lecture.

✓ Constraining the model is one of the two cures for variance, and the only one available when the dataset is fixed at 891 rows.

Ridge — the ℓ_2 penalty

$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) + \frac{\alpha}{m} \sum_{j=1}^{n} \theta_j^2$$

- The sum starts at $j = 1$: **the bias term θ_0 is not penalised**. Shrinking the intercept would pull every prediction towards zero rather than towards the mean
- Equivalently $\Omega(\boldsymbol{\theta}) = \frac{1}{2} \|\mathbf{w}\|_2^2$, where $\mathbf{w}$ is $\boldsymbol{\theta}$ without the bias

Also called Tikhonov regularisation; the book gives both names.

Ridge has a closed form too

Differentiate and set to zero, exactly as in Lecture 2:

$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^\intercal \mathbf{X} + \alpha \mathbf{A})^{-1} \mathbf{X}^\intercal y$$

$\mathbf{A}$ is the identity with a zero in the top-left cell — the identity minus the row and column belonging to the bias term.

THE TWO FAILURE CONDITIONS OF LECTURE 2, BOTH GONE

The normal equation is unsolvable when $m < n$ or when the columns are collinear. **Adding $\alpha \mathbf{A}$ removes both, for every $\alpha > 0$.** Ridge is not only a bias–variance device; it is what makes the linear algebra work.

The same term, on the log loss

$$J(\boldsymbol{\theta}) = -\frac{1}{m}\sum_{i}\left[y^{(i)}\log\hat{p}^{(i)} + (1-y^{(i)})\log(1-\hat{p}^{(i)})\right] + \frac{\alpha}{m}\|\mathbf{w}\|_2^2$$

Now the objective is **coercive**: as $\|\mathbf{w}\| \to \infty$ the penalty $\to \infty$ while the log loss is bounded below by zero. A minimiser exists, and strict convexity makes it unique.

✓ Diagnoses 2 and 3 are both gone, and we have not looked at the data once.

A trap in the API

`LogisticRegression` does not take α. It takes `C`.

$$C = \frac{1}{\alpha}$$

- Small `C` → large α → **strong** regularisation
- Large `C` → weak regularisation; `C=1e6` is what switches it off
- The default is `C=1.0` with `penalty="l2"`: **logistic regression in scikit-learn is regularised unless you say otherwise**

✗ The previous lecture turned it off on purpose, so that the coefficients it read were the data's. That choice is what left three faults to repair.

A penalty requires scaling. Not as a nicety

The penalty $\sum \theta_j^2$ treats every weight identically. If `Fare` is a price and `SibSp` is a count of people, a "weight of 1" means two unrelated things and the penalty falls unequally on them.

FAILURE CONDITION — AN UNSCALED PENALTY

Rescale a column by c and its fitted weight scales by $1/c$, so its contribution to the penalty scales by $1/c^2$. **Changing the unit of a column changes which coefficients ridge decides to shrink** — and nothing in the output says so.

✓ **`StandardScaler` is already inside the pipeline, fitted per fold. A penalty is the reason it is not optional.**

Lasso — the ℓ_1 penalty

$$J(\boldsymbol{\theta}) \;=\; \mathrm{MSE}(\boldsymbol{\theta}) \;+\; 2\alpha \sum_{j=1}^{n} |\theta_j|$$

Least Absolute Shrinkage and Selection Operator. The word **selection** is the point: lasso drives weights to exactly zero, so it performs feature selection as a side effect of fitting.

Ridge shrinks everything and eliminates nothing.

Why ℓ_1 produces exact zeros and ℓ_2 does not

Look at the gradient of the penalty near $\theta_j = 0$.

Ridge

$$\frac{\partial}{\partial \theta_j} \theta_j^2 = 2\theta_j \to 0$$

The push towards zero fades as you approach zero. You arrive only in the limit.

Lasso

$$\frac{\partial}{\partial \theta_j} |\theta_j| = \pm 1$$

The push keeps its size right up to zero. If the data pulls with less than α, the weight reaches zero and stops.

At $\theta_j = 0$ the absolute value is not differentiable; the subgradient is the whole interval $[-1, 1]$, and any data pull inside that interval is held at exactly zero.

Elastic net — both at once

$$J(\boldsymbol{\theta}) = \text{MSE} + 2r\alpha \sum_j |\theta_j| + \frac{(1-r)\alpha}{m} \sum_j \theta_j^2$$

r is the mix — `l1_ratio` in scikit-learn. $r = 0$ is ridge, $r = 1$ is lasso.

> *Prefer some regularisation to none. Ridge is a good default; if only a few features matter, prefer lasso or elastic net — and prefer elastic net to lasso when the features are correlated or outnumber the instances, because lasso behaves erratically there.*

✗ Both conditions hold here: 143 columns from 712 rows, and Age and Age^2 are about as correlated as two columns get.

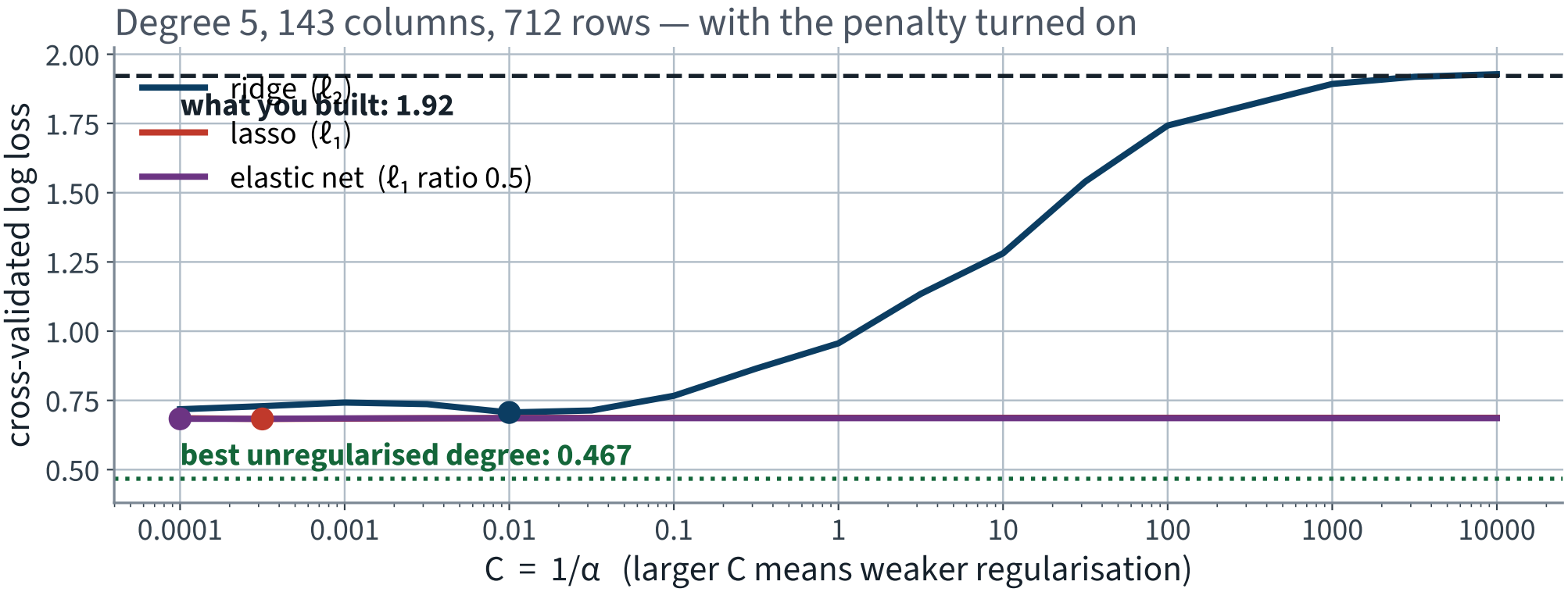
Sweep α on the degree-5 model that failed

```
Cs = np.logspace(-4, 4, 17)

for name, kw in (("ridge",   dict(penalty="l2", solver="lbfgs")),
                 ("lasso",   dict(penalty="l1", solver="saga")),
                 ("elastic", dict(penalty="elasticnet", solver="saga",
                                  l1_ratio=0.5, max_iter=3000))):
    for C in Cs:
        s = cross_val_score(pipeline(degree=5, C=C, **kw),
                            X_train, y_train, cv=cv,
                            scoring="neg_log_loss")
```

Three penalties, 17 values of `C`, ten folds each — three validation curves on one axis. Note the `solver=` changes: not every solver supports every penalty, and scikit-learn raises rather than silently ignoring the request.

What the penalty buys at degree 5



Every curve is far below the dashed line. None of them reaches the dotted one.

Reading it

Penalty at degree 5	Best `C`	Log loss	Non-zero weights
none — the previous lecture	—	✗ **1.922**	143
ridge	0.01	0.706	143
elastic net, $r = 0.5$	0.0001	0.684	3
lasso	0.00032	**0.683** ✓	3

Ridge alone removes **1.215** of log loss. Lasso removes **1.239** and throws away 140 of the 143 columns while doing it.

Both minima sit near the strong-penalty end of the range. Widen the grid before believing either — and the lasso curve is nearly flat there, so "the best `C`" is a plateau, not a point.

And now the uncomfortable part

Model	Cross-validated log loss
Degree 5, lasso, tuned	0.683
Degree 2, no penalty at all	**0.467 ✓**

✗ The best regularised degree-5 model does not beat the plain degree-2 model we already had.

Regularisation repaired an over-complex model. It did not turn it into a better one than the simple model, because there was never much bias to buy back — the decomposition said so twenty minutes ago, and this is what that sentence looks like in a table.

So what is a penalty actually for?

- **It makes the problem well-posed.** A unique minimiser exists for every $\alpha > 0$, whatever the columns do
- **It makes capacity a continuous dial.** Degree is an integer; α is not, so you can sit between two degrees
- **It is insurance.** When you cannot know the right capacity in advance — which is usually — it bounds the damage
- **It selects.** Lasso's three surviving columns are a hypothesis about which features matter, generated by the fit

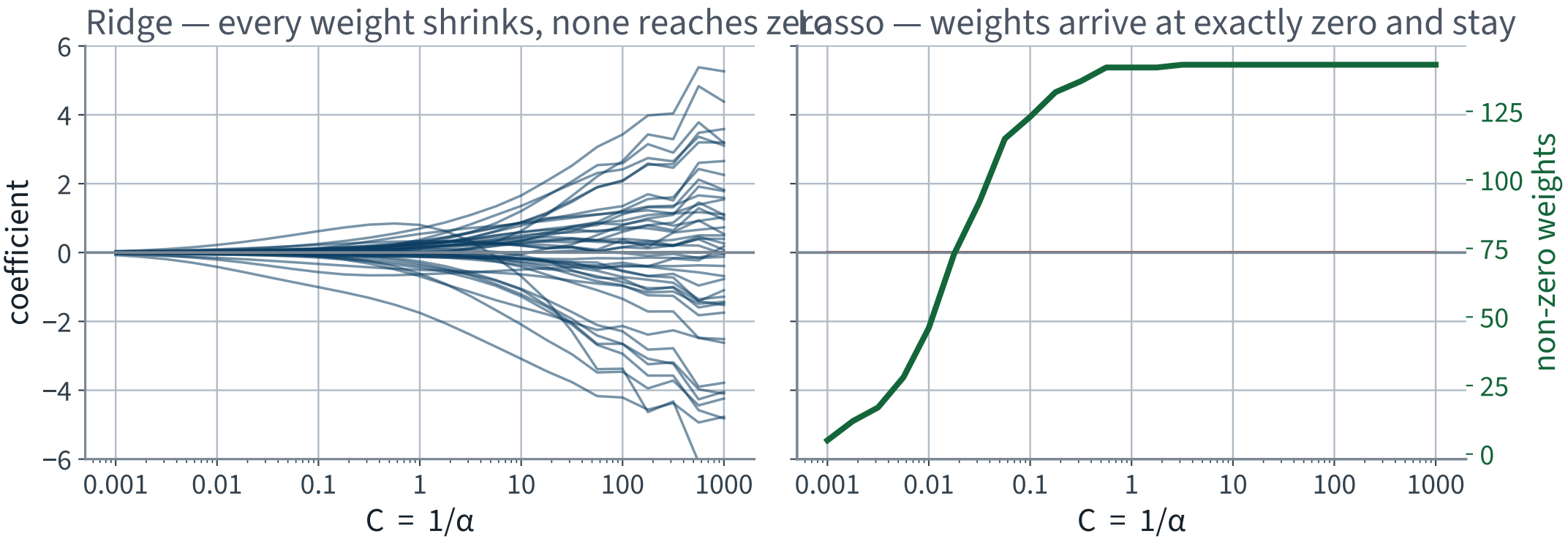
What it is *not* is a way to make an over-parameterised model beat a well-chosen one on a dataset this small.

Watch the weights as α moves

```
paths = []
for C in np.logspace(-3, 3, 25):
    m = pipeline(degree=5, C=C, penalty=pen, solver=sol).fit(X_train, y_train)
    paths.append(m[-1].coef_[0].copy())          # 143 weights per row
```

One row of 143 weights per value of `C`. Plot each weight against `C` and the two penalties become visibly different objects.

Ridge shrinks; lasso selects



Left: every path approaches zero and none arrives. Right: paths sit at exactly zero and leave one at a time as the penalty weakens.

Two things to read off those paths

- **At the strong-penalty end both panels are flat near zero.** That is the high-bias regime: the model is being told to ignore the data, and it obeys
- **At the weak-penalty end the ridge paths fan out to ± 6 and beyond.** That is diagnosis 2 reasserting itself as the penalty stops holding it down

> Across six decades of `C` the lasso count rises from 5 non-zero weights to 142 of the 143. Every point on that line is a different model, and cross-validation is what picks one.

Early stopping — a penalty by another route

Fit iteratively, and **stop before the optimiser gets where it is going**.

- Gradient descent starts at $\boldsymbol{\theta} = \mathbf{0}$ — the smallest possible model
- Each epoch moves the weights outward, fitting more of the training set
- Held-out error falls, reaches a minimum, and then rises

✓ Géron calls it a "beautiful free lunch": no penalty term and no extra term in the objective — just a stopping rule. The knob is the epoch count, and it is a hyperparameter like any other.

In code, one epoch at a time

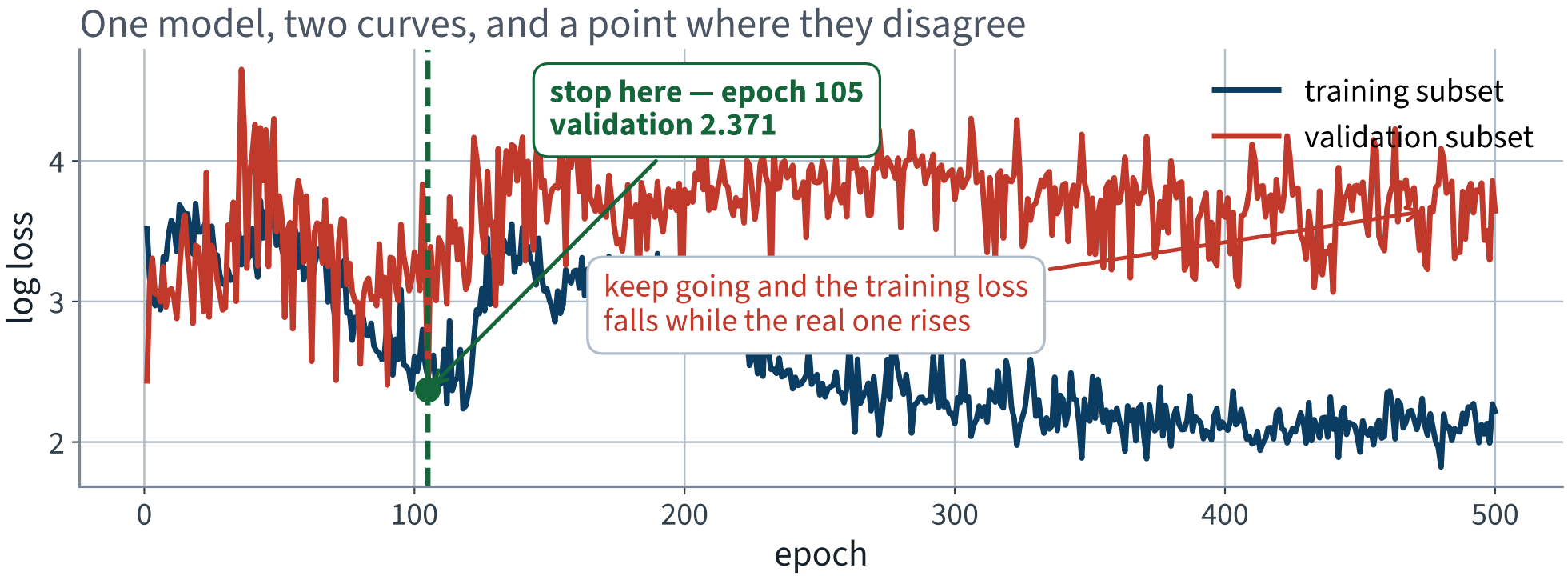
```
clf = SGDClassifier(loss="log_loss", penalty=None, learning_rate="constant",
                    eta0=0.0015, random_state=42,
                    warm_start=True, max_iter=1, tol=None)

for epoch in range(500):
    clf.fit(Z_train, y_a)                # warm_start: resume, do not restart
    train.append(log_loss(y_a, clf.predict_proba(Z_train)[:, 1], labels=[0, 1]))
    valid.append(log_loss(y_b, clf.predict_proba(Z_valid)[:, 1], labels=[0, 1]))
```

`warm_start=True` with `max_iter=1` is what makes each call one epoch of the *same* fit. `penalty=None` is deliberate: early stopping should be the only regularisation in the room.

The transform is fitted on the training part only, outside the loop, and applied to both. Refitting it inside the loop would leak, 500 times over.

The point where the two curves disagree



Epoch 105 is the best model this run ever holds. Running to epoch 500 costs 1.274 of log loss and never announces it.

Reading it honestly

	Epoch 105	Epoch 500
Training log loss	2.458	2.219
Validation log loss	**2.371 ✓**	**✗ 3.645**

- These are bad numbers in absolute terms. Plain SGD at a constant learning rate on 143 correlated columns is a poor optimiser, and this shows *the shape*, not a competitive model
- The training curve is noisy — each epoch reshuffles — so "stop at the minimum" needs a patience rule in practice, not a single-epoch comparison

✗ The stopping epoch is chosen by looking at held-out data. It must be chosen on validation rows, never on the test set.

Four repairs, one idea

Repair	Keeps $\|\theta\|$ small by	Zeros?
Ridge	penalising $\sum \theta_j^2$	**no ✗**
Lasso	penalising $\sum \lvert\theta_j\rvert$	**yes ✓**
Elastic net	both, mixed by r	**yes ✓**
Early stopping	starting at zero and not going far	**no ✗**

All four reduce variance by shrinking the set of models the fit is allowed to reach. All four increase bias. Which is why all four need α — or an epoch count — chosen by cross-validation.

FURTHER GROUND

Choosing α, and the test set once

15 minutes · examinable

Choosing a hyperparameter is choosing a model

```
best_C, best_score = None, np.inf

for C in np.logspace(-4, 4, 17):
    m = pipeline(degree=3, C=C).fit(X_train, y_train)
    score = log_loss(y_test, m.predict_proba(X_test)[:, 1])
    if score < best_score:
        best_C, best_score = C, score
```

FAILURE CONDITION — A HYPERPARAMETER CHOSEN ON THE ROWS YOU THEN REPORT

`best_score` is a **minimum over seventeen noisy estimates**, and the minimum of noisy estimates is biased downward even when every estimate is individually unbiased. The code never writes to the test set. It *reads* it seventeen times, and reading is enough.

Measure it rather than asserting it

20 splits. On each: run the loop above, and also the honest version that picks `C` by 5-fold cross-validation *inside* the training part. Score both on the same held-out rows.

Reported number	Log loss	Spread over 20 splits
Chosen on the test set, scored on the test set	✗ **0.451**	0.049
Chosen by cross-validation, scored on the test set	**0.470** ✓	0.045

Optimism: 0.019 of log loss, about 4.1%. On 13 splits of 20 the first number is the flattering one, and the two procedures picked the same `C` on 7 splits of 20.

Read that result carefully — both halves

It is small.

0.019 against a split-to-split spread of 0.045. On one split you would never see it. Seventeen candidates is a small search.

It is real, and one-sided.

13 splits of 20 flattered. It never averages away, and it grows with the number of candidates you try.

THE SCALING RULE

Small search, small bias. Search a thousand configurations — which a randomised search does in an afternoon — and the same procedure hands you a number that is confidently wrong. **The size of the error scales with how hard you looked.**

The procedure that does not have the problem

```
gs = GridSearchCV(pipeline(degree=3), {"clf__C": np.logspace(-4, 4, 17)},
                  scoring="neg_log_loss", cv=cv).fit(X_train, y_train)

print(-gs.best_score_)                                  # the honest estimate
print(log_loss(y_test, gs.predict_proba(X_test)[:, 1]))   # the test set, once
```

Same seventeen candidates, same runtime, **two numbers instead of one**. Report both. The second is allowed to be worse than the first, and if it is much worse that is itself the finding.

The standing rule, extended by eleven words: nothing derived from the test set may appear in the training path — including the choice of any hyperparameter.

The test set. Once

```
for name, m in candidates.items():
    m.fit(X_train, y_train)
    p = m.predict_proba(X_test)[:, 1]
    print(name,
          log_loss(y_test, p, labels=[0, 1]),
          brier_score_loss(y_test, p),
          accuracy_score(y_test, p >= 0.5))
```

179 passengers, untouched since the split in the previous lecture. Five candidates, all of whose hyperparameters were fixed before this cell ran.

The honest numbers

Model	Log loss	Brier	Accuracy
Degree 5, no penalty — where the sweep ended	**✗ 1.727**	0.189	74.9%
Degree 5, ridge, tuned	0.804	0.173	77.1%
Degree 5, lasso, tuned	0.677	0.244	65.9%
Degree 1, scikit-learn defaults	0.433	0.134	**82.7% ✓**
Degree 2, no penalty — **the winner**	**0.427 ✓**	**0.134 ✓**	**82.7% ✓**

The winner improves on the degree-5 model the previous lecture finished with by **1.300** of log loss.

The winner is a model with no repair in it

Degree 2, no penalty, chosen by reading a held-out curve.

Row four deserves as much attention: `LogisticRegression()` with every default left alone — degree 1, `C=1.0`, `penalty="l2"` — scores 0.433. Statistically it is the same model as the winner.

> **THE RESULT NOBODY WANTED**
>
> Ninety minutes of ridge, lasso, elastic net and early stopping, and the best system on the test set is a two-line model anyone could have written before the previous lecture started.

✓ Report it anyway. The alternative is choosing the interesting model over the better one.

Then what was any of it for?

- You now know **why** degree 2 wins, and can say it in three terms rather than as a preference
- You know the floor is **0.121** Brier and that no model of these columns goes below it
- You know that the degree-5 model was not merely worse but *ill-posed*, and exactly which term repairs that
- You have four repairs, measured, for the next dataset — where 143 columns will arrive with a hundred times as many rows and the answer will be different

A negative result you can explain is a result. A positive result you cannot explain is a liability.

Is 0.427 good?

The brief asked for calibrated probabilities and a defensible cut-off, not for a headline. Against that brief:

- Cross-validated, the model beats the anchor by **0.199** of log loss and the majority class by **20.5** points of accuracy. Both are real
- Brier 0.134 against a measured floor of about 0.121. **There is roughly 0.013 of Brier score left on the table**, in total, for any model of these columns
- 179 test passengers, so a difference of a point or two of accuracy is three or four people and is not a difference

✓ That third bullet belongs in the report. A test set of 179 cannot support the comparisons people will want to make with it.

Regularisation checklist — keep this

- Is the data scaled, *inside* the pipeline, before a penalty is applied?
- Is the bias term excluded from the penalty?
- Was α (or `C`) chosen on validation folds, never on the test set?
- If the search sat at the edge of the grid, was the grid widened?
- Is the unregularised model in the comparison table, so the penalty has to earn its place?
- Was the stopping epoch, if any, chosen on held-out rows?

Six questions. The fifth is the one people skip, and it is the one that would have saved us an hour today.

The notebook for this lecture

Open Lecture 5 in Colab

- **Runs on free CPU** in about four minutes. The slowest cell is the 200-resample decomposition and it says so
- It rebuilds the degree sweep from scratch, so you can reproduce 1.922 before repairing it — then measures all three terms and checks the identity closes to 10^{-17}
- Then the noise floor, the learning curves, the three penalty sweeps, the coefficient paths, early stopping, and the test set once

> **WHAT TO CHANGE**
>
> In the penalty sweep, widen `np.logspace(-4, 4, 17)` to `(-8, 4, 25)`. Does the lasso minimum move? If it does, the number on the slide above was an artefact of where we stopped looking, and you have just done checklist item four.

Every notebook in the course

01 · What machine learning is, and how we will work

02 · The end-to-end project

03 · Classification and its metrics

04 · Training models

05 · Regularisation and the bias–variance trade-off

06 · Decision trees

07 · Ensembles and random forests

08 · Dimensionality reduction and unsupervised learning

09 · Neural networks, from the perceptron up

10 · PyTorch

11 · Training deep networks

12 · Convolutional networks

13 · Transfer learning

14 · Detection and segmentation

15 · Time series

16 · Recurrent networks

17 · Text

18 · Attention and transformers

19 · Information retrieval: the lexical foundation

20 · Information retrieval: dense retrieval

21 · Recommender systems: from ratings to factors

22 · Recommender systems: neural, and evaluated honestly

23 · Vision transformers and multimodal retrieval

24 · Generation, retrieval-augmented systems, and where this leaves you

Summary

1. Expected squared error splits exactly into squared bias, variance and irreducible noise — over the training set and the label, not over test points
2. The gap between a training curve and a held-out curve *is* the variance term; curves plateauing together mean bias instead
3. The noise floor is measurable: 0.121 Brier, from passengers whose recorded values are identical
4. Degree 5 failed three ways at once — high variance, no minimiser, and no unique minimiser
5. Adding $\alpha\mathbf{A}$ repairs the last two for every $\alpha > 0$, which is Lecture 2's open question answered
6. Ridge shrinks, lasso selects, elastic net does both, early stopping does it by not arriving
7. None of them beat choosing the right capacity. A penalty is a repair, not an upgrade
8. A hyperparameter chosen on the test set costs 0.019 of log loss here, and the cost grows with the size of the search

In the book

Topic	Chapter
Learning curves; the bias/variance trade-off	4
Ridge, lasso, elastic net	4
Early stopping	4
Regularised logistic regression; `C`	4
The normal equation and its invertibility	4
Cross-validation and grid search	2
Overfitting, underfitting, generalisation error	1

The noise-floor measurement by identical-input cells is . The decomposition itself, and the BEYOND THE SYLLABUS, FOR CONTEXT eigenvalue argument for $\alpha\mathbf{I}$, are examinable as derived here.

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 5

five, in the style of the written paper

Exercises — Lecture 5

1 Write the bias–variance decomposition of the expected squared error, and name the one term no model can reduce. [4]

2 A learning curve shows training and validation error both high and close together, and flat as data is added. Diagnose it, and say whether more data would help. [5]

3 Ridge adds αI to $X^\top X$ before inverting. Give the numerical consequence, in terms of the condition number. [4]

Solutions on Lecture 6's deck.

Exercises — Lecture 5 (continued)

4 Why must features be scaled before ridge or lasso, when they need not be for ordinary least squares? [4]

5 You tune α over a grid using cross-validation, then report the best cross-validated score as your estimate of future performance. Name the error, and quantify its direction. [5]

Solutions on Lecture 6's deck.

THE FIVE SET AT THE END OF LECTURE 4

Solutions Lecture 4

not part of this lecture

Solutions — Lecture 4

1. Gradient descent on the same data, with the features unscaled, takes many more steps to converge than on…

✓ Unscaled features make the contours elongated, so the gradient points across the valley rather than along it.

- The curvature in each direction is proportional to that feature's scale, so a large-scale feature dominates the step
- The learning rate must then suit the steepest direction, which makes it far too small for the shallow one

2. Your logistic regression on one-hot encoded columns produces coefficients that look wrong, and the design…

✓ Each fully encoded categorical adds a redundant column; drop one level per categorical.

- The indicator columns of one categorical sum to the intercept column, which is an exact linear dependence
- With a reference level dropped the coefficients become differences from that level, and are interpretable again

Solutions — Lecture 4 (2)

3. A model reaches 0.80 accuracy and is badly calibrated. Explain what that means, and name a decision it would…

✓ Its predicted probabilities do not match observed frequencies; any decision using a cost-weighted threshold is then wrong.

- Accuracy reads only the arg-max, so it is blind to the probability that produced it
- Choosing a threshold from expected cost requires the probability to mean what it says

4. Why does the log loss have no closed-form minimiser, when squared error does?

✓ Its gradient is not linear in the parameters, so setting it to zero gives no solvable system.

- Squared error is quadratic in θ, so its gradient is affine and the stationary condition is a linear system
- The sigmoid makes the log-loss gradient transcendental in θ

Solutions — Lecture 4 (3)

5. State the update rule for batch gradient descent, and say what changes in it for stochastic gradient descent…

✓ $\theta \leftarrow \theta - \eta \nabla J$. Only the set the gradient is computed over changes; the rule does not.

- SGD estimates the same gradient from one instance rather than all of them
- The estimate is unbiased and noisy, which is why the step size usually has to decay

NEXT LECTURE

Decision trees

A model that is not linear in anything, needs no scaling, and whose failure condition is that it is — which is the variance term of today, in a new shape